

Nexus Ciel — Documentation Technique de Construction (V2.5.2 → V1 build-ready)

Nexus Ciel — Documentation Technique de Construction

Version : V2.5.2 consolidée → spécification de construction V1

Public visé : agent de code (Cursor, Claude, etc.) et développeurs humains.

But du document : décrire Nexus Ciel de façon assez complète et non ambiguë pour qu'un agent de code puisse le **construire entièrement**, sans avoir à improviser une seule décision d'architecture.

0. Comment lire et utiliser ce document

Ce document est la **source de vérité** de la construction. Il est organisé en pages :

1. **Vision & principes fondateurs** (cette page)
2. **Décisions d'architecture — résolution des frictions F1→F7** (les 7 choix tranchés)
3. **Modules du noyau — les 11 modules principaux** (spécifications détaillées)
4. **Modules de support — les 16 modules périphériques**
5. **Architecture API & fournisseurs**
6. **Schémas de données & contrats** (manifeste de capacité, fiche de savoir-faire, état de mission...)
7. **Structure du dépôt & plan de construction par phases**
8. **Guide de l'agent de code (Cursor / LLM) — règles à respecter pendant le build**

Règle d'or pour l'agent de code : si une décision n'est pas écrite ici, **ne l'invente pas** — signale le trou, propose 2 options, et attends. Le but explicite de ce document est d'empêcher l'improvisation.

1. Vision

Nexus Ciel est une **entité logicielle autonome** (pas un simple script d'agent) qui :

- reçoit des **missions**, les planifie, les exécute et les valide ;
- **apprend** de chaque mission (transforme l'expérience en savoir-faire réutilisable) ;
- **crée les capacités qui lui manquent** quand une mission révèle un manque ;
- **économise** agressivement : elle n'appelle un grand modèle que lorsque rien de moins cher ne suffit ;
- **évolue** de façon mesurée et prouvable, sans jamais modifier son propre noyau ;
- **agit le jour, digère la nuit** : l'apprentissage se fait en tâches de fond pendant l'inactivité.

La distinction fondatrice : **noyau gelé** (immuable, humain seul peut le changer) vs **capacités & politiques** (territoire d'évolution de Nexus, traitées comme des *données*, pas comme du code système).

2. Les 6 principes fondateurs (à respecter partout)

P1 — Activation paresseuse

Aucun module hors noyau n'est actif par défaut. Chaque module déclare ses **conditions de réveil** (abonnements à des événements de l'Event Bus). Manas Core arbitre les activations.

Noyau minimal (toujours actif) : Manas Core · Event Bus · State Graph · Capability Registry · Execution Fabric · Validation Engine · Mission Journal · Mission Guard · AI Gateway. **Tout le reste dort.**

P2 — Source de vérité unique

Chaque donnée a **un seul** propriétaire. Registry (capacités), Mission Journal (historique), State Graph (état de mission) sont les sources de vérité. Le Memory Core **n'entrepasse pas** ces données : il les **indexe** (embeddings, cache sémantique, liens) pour la récupération. La mémoire est une *couche d'accès*, pas un second entrepôt.

P3 — Contrat de noyau gelé

Le noyau est **gelé** : modifiable uniquement par l'humain. Les capacités et politiques (routage, stratégies, mémoire) sont le territoire d'évolution de Nexus. Frontière matérialisée dans le code : **répertoires séparés**, noyau en **lecture seule** pour l'Execution Fabric.

P4 — Statut probatoire généralisé

Toute nouveauté (capacité créée, fusion, méthode assimilée, politique évoluée) naît **probatoire** et n'est promue **éprouvée** qu'après **N succès réels** en mission. Un seul mécanisme uniforme qui protège contre les 3 risques : hallucination d'outil, sur-généralisation, dérive d'évolution.

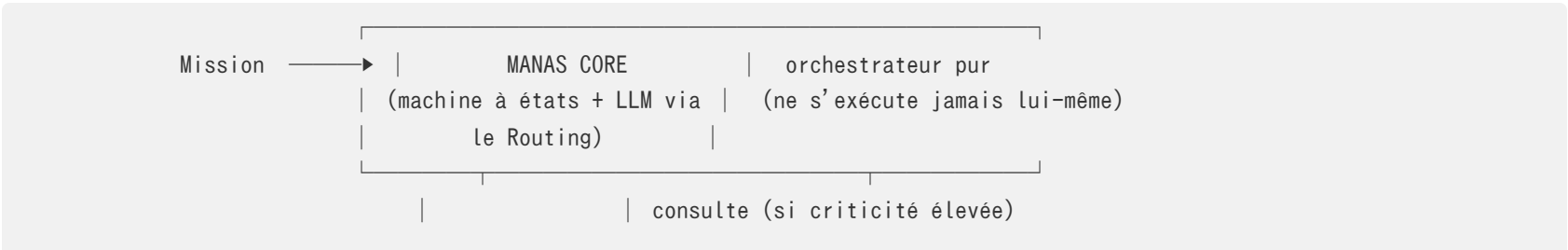
P5 — Cycle de digestion (le « sommeil »)

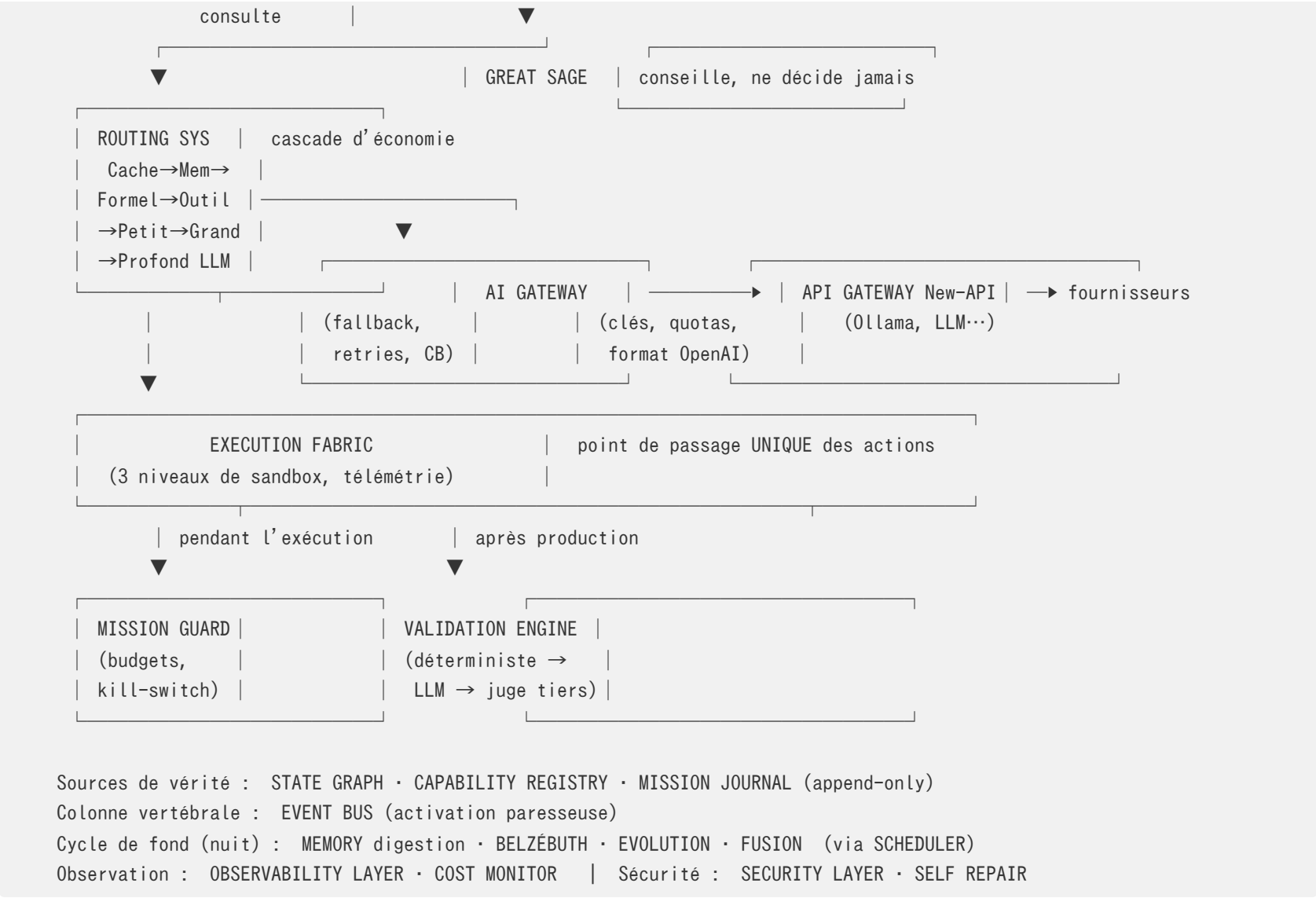
Tous les processus d'apprentissage (consolidation mémoire, Belzébut, Evolution, Fusion) sont regroupés dans un **cycle de fond** orchestré par le Scheduler pendant l'inactivité. L'apprentissage ne concurrence jamais les missions en ressources.

P6 — Validation en couches, jamais de juge unique

On valide d'abord par des moyens **déterministes** (schéma, contraintes, tests, raisonnement formel), puis par un LLM seulement pour l'invérifiable mécaniquement, et pour le critique par un **fournisseur différent** de celui qui a produit (décorrélation des erreurs). Tout se termine par des **budgets de retries/replanifications** pour éviter les boucles infinies.

3. Architecture globale (vue d'ensemble)





Flux nominal d'une mission :

1. Mission reçue → Manas crée un nœud dans le **State Graph**, ouvre une entrée dans le **Mission Journal**.
2. Manas calcule un **score de criticité**. Si > seuil → consulte **Great Sage**.
3. Manas (aidé du **Planner**) construit des plans candidats, en choisit un.
4. Chaque action passe par le **Routing** (cascade d'économie) puis l'**Execution Fabric**.
5. Le **Mission Guard** surveille en temps réel (budgets, périmètre, actions interdites).
6. Chaque résultat passe par le **Validation Engine** (couches déterministe → LLM → juge tiers).
7. À la fin : rapport (**Reporting Engine**), puis **cycle de digestion** (Belzébuth apprend, Memory consolide) déclenché par le Scheduler à l'inactivité.

(Voir les pages suivantes pour le détail de chaque décision et module.)

1. Décisions d'architecture — résolution des frictions

F1 → F7

Décisions d'architecture — les 7 frictions tranchées

Ces 7 décisions sont **figées**. Elles résolvent les incohérences structurelles de la V2.5.2. L'agent de code doit les traiter comme des contraintes dures.

F1 — Qui apprend le routage ? → l'Evolution Engine, seul

Problème : le Routing « apprend » ET l'Evolution Engine « améliore le routage » = deux propriétaires du même apprentissage = oscillations et écrasements.

Décision :

- Le **Routing** *collecte* les statistiques (succès/échec, coût, latence par étage) et *applique* la politique en vigueur. **Il n'apprend rien en direct.**
- L'**Evolution Engine** est le **seul** à modifier la politique de routage : **hors ligne, par lots, après validation sur le Gymnase**, avec possibilité de rollback.

Implémentation : la politique de routage est un artefact versionné (fichier/enregistrement) en lecture seule pour le Routing, en écriture seule pour l'Evolution Engine.

F2 — Great Sage vs économie → score de criticité + seuil de réveil

Problème : confronter plusieurs points de vue à chaque décision = multiplication d'appels LLM, contraire à la cascade d'économie.

Décision : définir un **score de criticité** :

$$\text{criticité} = \text{coût_estimé} \times \text{irréversibilité} \times \text{incertitude}$$

- Chaque facteur est normalisé sur [0,1]. Seuil de réveil configurable (défaut proposé : **0.6**).
- Au-dessus du seuil** : Great Sage se réveille (multi-perspectives).
- En dessous** : Manas décide seul. Cohérent avec l'activation paresseuse (P1).

F3 — Place du raisonnement dans la cascade → **séparer formel et profond**

Problème : la cascade terminait par « Grand modèle → Raisonnement profond », mais le raisonnement **formel** (SAT/SMT/Z3) est déterministe et peu coûteux — il ne doit pas être le dernier recours.

Décision — cascade officielle (ordre du moins cher au plus cher) :

1. Cache sémantique
 2. Mémoire (Memory Core, récupération indexée)
 3. Algorithme classique / Raisonnement FORMEL (SAT/SMT/Z3, déterministe)
 4. Outil spécialisé (capacité du Registry)
 5. Petit modèle LLM
 6. Grand modèle LLM
 7. Raisonnement PROFOND LLM (extended thinking, chaînes longues) — le plus cher

- **Raisonnement formel = niveau 3. Raisonnement profond LLM = niveau 7.** Ne jamais les confondre.

F4 — Triple stockage → **source de vérité unique (voir P2)**

Problème : « Mémoire des capacités » vs Capability Registry ; « Mémoire des missions » vs Mission Journal ; « Scratchpad » vs State Graph = 3 paires de doublons.

Décision :

Donnée	Source de vérité (unique)	Rôle du Memory Core
Capacités	Capability Registry	indexe descriptions (embeddings) pour retrouver la bonne capacité
Historique missions	Mission Journal (append-only)	indexe pour rappel contextuel de missions similaires
État de mission	State Graph	le Scratchpad n'est qu'une <i>vue de travail</i> , pas un second état

Le Memory Core **indexe et met en cache**, il **n'entrepose jamais** la donnée canonique.

F5 — Self Repair vs « jamais modifier son propre code » → périmètre strict

Décision — Self Repair a le droit de :

- redémarrer des composants ;
- faire un **rollback** vers une version connue-bonne ;
- réparer configuration / état corrompu ;
- réinstaller des dépendances.

Self Repair n'a **jamais** le droit d'**éditer le code du noyau**. Les capacités *créées par Nexus* (hors noyau) peuvent être régénérées : ce sont des **données**, pas du code système. Cohérent avec le Contrat de noyau gelé (P3).

F6 — AI Gateway vs API Gateway → deux couches distinctes

Décision :

	API Gateway (New-API/One-API)	AI Gateway (couche Nexus)
Nature	Composant sur étagère	Code Nexus maison
Rôle	Agrégation fournisseurs, clés, quotas, format unifié OpenAI-compatible	Retries, fallback entre fournisseurs, timeout, circuit breakers, télémétrie par appel, application des décisions du Routing
Connaissance des missions	Non	Oui

Choix retenu : **New-API** (plus actif que One-API). L'AI Gateway s'appuie dessus.

F7 — Mission Guard vs Validation Engine → pendant vs après

Décision :

- **Mission Guard** = contrôle **temporel, en temps réel, PENDANT** l'exécution : budget, temps, périmètre, actions interdites, kill-switch. **Il interrompt.**
- **Validation Engine** = jugement **APRÈS** production d'un résultat : qualité, cohérence, preuves, objectifs atteints. **Il juge.**

Règle mnémotechnique : *le Guard coupe, la Validation note.*

2. Modules du noyau — les 11 modules principaux

Modules du noyau — spécifications détaillées

Pour chaque module : **rôle**, **contrat (entrées/sorties)**, **décisions figées**, **cas limites à gérer**.

1. Manas Core — l'orchestrateur

Rôle : orchestrateur pur. Il décide, il ne s'exécute jamais lui-même.

Nature : **hybride**. Squelette = **machine à états déterministe** opérant sur le State Graph ; il *consulte* des modèles via le Routing pour les jugements. **Le squelette n'est jamais probabiliste**.

Frontière avec le Planner : le **Planner est un outil de Manas** (génère et score des plans candidats). Manas garde la décision finale.

Critère de « mission terminée » : checklist d'objectifs validés par le **Validation Engine** (jamais un « sentiment » de LLM).

Cas limites :

- Mission impossible/contradictoire → **abandon propre avec rapport**, pas de boucle.
- **Budget de replanifications max** (**M** , voir Validation Engine) : au-delà, abandon propre.

Contrat :

- *Entrée* : mission (objectif, contraintes, budgets).
- *Sortie* : suite d'actions déléguées à l'Execution Fabric + décision finale (succès / abandon) + entrées Journal.

2. Great Sage Layer — le conseil multi-perspectives

Rôle : réduire les hallucinations sur les décisions critiques. **Conseille, ne décide jamais**.

Déclenchement : uniquement quand **criticité > seuil** (voir F2).

Les 5 perspectives — mix explicite, pas 5 LLM systématiques :

Perspective	Implémentation
Analyse	appel LLM

Mémoire	requête au Memory Core (pas un LLM)
Critique	appel LLM
Simulation	appel LLM (ou moteur si applicable)
Stratégie	appel LLM

Format du consensus : recommandation + niveau de confiance + objections non résolues . Manas doit voir les désaccords, pas seulement la synthèse.

Cas limite : pas de consensus → remonter les deux options avec arguments à Manas. Jamais de consensus artificiel forcé.

3. Adaptive Intelligence Routing System — le cœur économique

Rôle : pour chaque besoin, descendre la cascade d'économie (voir F3) et s'arrêter dès qu'un étage suffit.

Critère de suffisance par étage : chaque étage produit un score de confiance. Si < seuil_étage → escalade automatique. Le Validation Engine renvoie du feedback qui calibre ces seuils (via l'Evolution Engine, jamais en direct — F1).

Démarrage à froid : livrer une politique par défaut écrite à la main (table type de tâche → étage), remplacée progressivement par l'Evolution Engine.

Cas limite critique — le cache sémantique qui renvoie « proche mais faux » :

- seuil de similarité strict ;
- tag de fraîcheur sur chaque entrée ;
- invalidation explicite.

4. Memory Core — couche d'accès + politique d'oubli

Rôle : indexer et retrouver (voir P2/F4). N'entrepasse pas les données canoniques.

Le vrai enjeu = politique d'oubli. Sans elle, la mémoire dégrade le système.

Chaque souvenir porte : score d'utilité (initialisé par Belzébut) · compteur d'usage · date de dernier accès · provenance .

Cycle de digestion (tâche de fond, Nexus inactif) :

- consolide : mémoire courte → longue si réutilisée ;
- déclasse : le score décroît sans usage ;

- **archive : jamais de suppression brute** — archivage froid consultable.
- La **mémoire procédurale** n'est promue qu'après **N réutilisations réussies validées**.

Cas limite : deux souvenirs contradictoires → garder les deux (horodatage + provenance), le plus récent prime par défaut, conflit signalé à Manas s'il est sollicité.

5. Belzébut Assimulation Engine — expérience → savoir-faire

Rôle : transformer l'expérience en savoir-faire réutilisable. **Déclencheur** : **fin de mission** (activation paresseuse).

Sortie obligatoire = fiche de savoir-faire (schéma détaillé dans la page « Schémas de données ») : **contexte** déclencheur, méthode, pièges connus, coût typique, taux de réussite observé. **Pas de texte libre**.

Assimiler les échecs autant que les réussites (anti-patterns : « ne pas refaire X dans le contexte Y »).

Risque assumé — sur-généralisation à partir d'une seule mission : une méthode extraite reste **hypothèse** jusqu'à confirmation par une **2e mission**. Le Routing la traite avec prudence.

6. Capability Creation Engine — ● le point le plus dangereux

Rôle : créer les capacités manquantes. Cœur de la vision, mais zone de recherche.

Problème résolu : « testée par qui, contre quoi ? » Si le même LLM génère l'outil ET son test, il valide ses propres hallucinations.

Pipeline obligatoire :

1. **Le besoin est défini par Manas AVANT génération** : spécification + **exemples d'entrées/sorties réels** tirés de la mission qui a révélé le manque. **Le test existe avant le code** (le blocage réel EST l'oracle).
2. **Génération** par un modèle, **revue par un AUTRE modèle** (ou le Formal Reasoning quand applicable).
3. **Exécution en sandbox stricte isolée** (obligatoire, sans réseau par défaut) sur les exemples réels.
4. **Intégration au Registry en statut** **probatoire** ; promotion **éprouvée** après N usages réussis en mission.

Cas limite : échec répété de génération → Manas conclut « capacité hors de portée actuelle », journalise, cherche une stratégie alternative. **Pas de boucle**.

7. Capability Fusion Engine — ● à garder, à activer en dernier

Rôle : lutter contre l'explosion du Registry en fusionnant des capacités proches. **Module le plus spéculatif** → dernière phase.

Détection de proximité : embeddings des descriptions + recouvrement des signatures E/S + co-occurrence dans les missions.

Politique de dépréciation (obligatoire, sinon le Registry triple) : la fusion devient la version par défaut ; les originaux passent en **legacy**, invisibles au Routing **sauf** échec de la fusion.

Règle : une fusion est une **nouvelle capacité** → elle repasse par **tout le pipeline du Creation Engine** (tests, probatoire).

8. Evolution Engine — amélioration mesurée et prouvable

Rôle : améliorer les politiques (routage, stratégies). **Seul propriétaire de l'apprentissage du routage** (F1).

Contre quoi mesurer ? → **le Gymnase de missions** (voir S2) : jeu de missions de référence rejouables. Toute modif de politique est **validée sur le Gymnase avant application** et **annulable (rollback)** si les métriques réelles se dégradent.

Fréquence : par lots, hors ligne, **jamais en cours de mission**.

Traçabilité : chaque changement appliqué = **entrée signée dans le Mission Journal**.

Cas limite — dérive lente : comparaison périodique contre la **politique par défaut d'origine**, gardée comme étalon.

9. Formal Reasoning Layer — excellent, sous-exploité

Rôle : résolution déterministe (SAT/SMT/Z3). Différenciateur face aux archis « tout-LLM ». **Niveau 3 de la cascade** (F3).

Extension : sert aussi de **vérificateur** pour le Validation Engine (contraintes logiques, cohérence de plans, vérification de code généré par analyse statique).

Qui traduit en SAT/SMT ? Souvent un LLM — **acceptable**, car la *résolution* reste déterministe et le résultat vérifiable.

10. Execution Fabric — point de passage unique

Rôle : **toutes** les actions passent par ici → point d'observation et de contrôle unique (parfait pour Mission Guard + Observability).

3 niveaux d'isolation :

Niveau	Cible	Sandbox
--------	-------	---------

a	Code du noyau	hors sandbox
b	Outils éprouvés du Registry	sandbox légère
c	Code fraîchement généré (Creation Engine)	sandbox stricte, sans réseau par défaut

Effets de bord : distinguer actions **réversibles** / **irréversibles**. Les irréversibles déclenchent Mission Guard (et éventuellement Great Sage).

Robustesse : timeout + retry **par type d'action** ; télémétrie systématique vers l'Observability Layer.

11. Validation Engine — juge en couches (voir P6)

Rôle : juger un résultat APRÈS production (F7). **Éviter le juge unique LLM**.

3 couches :

1. **Vérifications déterministes d'abord** : schéma, contraintes, tests, Formal Reasoning.
2. **Jugement LLM** uniquement pour l'invérifiable mécaniquement.
3. **Si critique** : juge d'un **fournisseur différent** de celui qui a produit (décorrélation des erreurs).

Budgets anti-boucle : max **N** tentatives avant replanification forcée ; max **M** replanifications avant abandon propre.

Boucle d'apprentissage : le feedback de validation alimente **Belzébuth** et l'**Evolution Engine**.

3. Modules de support — les 16 modules périphériques

Modules de support — les 16 modules périphériques

Les modules du **noyau minimal (toujours actifs, P1)** sont marqués . Le reste dort et se réveille sur événement.

Passerelles & fournisseurs

AI Gateway (noyau)

Couche Nexus au-dessus de l'API Gateway (F6). Responsabilités : **fallback entre fournisseurs, retries, timeout, circuit breakers, télémétrie par appel**, application des décisions du Routing. Connait les missions.

API Gateway (New-API / One-API)

Composant **sur étagère** : agrégation des fournisseurs, clés, quotas, format OpenAI-compatible unifié. **Choix retenu : New-API** (plus actif). Ne connaît pas les missions.

GPT4Free Integration

OK en interne, mais **confiné** : tag `fournisseur non fiable`, **jamais sélectionné pour missions critiques ni pour la validation**. Ses stats de fiabilité sont tracées séparément.

Planification & orchestration de fond

Planner

Outil de Manas (décision retenue, voir §1 noyau) : génère et score des plans candidats. Ne décide pas.

Scheduler

Déclenche les **cycles de fond** (digestion mémoire, Belzébuth, Evolution, Fusion) pendant l'inactivité — c'est le « **sommeil** » de l'entité (P5).

Event Bus (noyau)

Colonne vertébrale de l'activation paresseuse (P1) : les « conditions de réveil » des modules sont des **abonnements à des événements**. À documenter comme infrastructure centrale.

État, gardes & traçabilité

State Graph (noyau)

Source de vérité de l'état de mission (F4). Le Scratchpad du Memory Core n'est qu'une **vue de travail**. Doit permettre la **reprise après crash**.

Mission Guard (noyau)

Contrôle temps réel (F7). Ajouts obligatoires : **budgets durs** (coût, temps, appels) par mission, **kill-switch global**, **liste d'actions interdites**.

Mission Journal  (noyau)

Append-only (jamais modifiable) : preuve d'exécution et matière première de Belzébuth. Source de vérité (F4).

Reporting Engine

Consommateur du Journal. Définir **3 formats** : rapport de mission, rapport d'évolution, rapport de coûts.

Registres de capacités

Tool Registry → **fusionné**

Décision retenue : **fusionner Tool Registry et Capability Registry en un seul Capability Registry**, où « outil » est **un type de capacité parmi d'autres** (script, workflow, agent, prompt-recette). Deux registres = double maintenance sans bénéfice.

Capability Registry  (noyau)

LE contrat central du système. Définir le **manifeste de capacité** (schéma complet dans la page « Schémas de données ») : `nom, description, signature E/S, coût typique, statut (probatoire/éprouvé/legacy), stats d'usage, conditions de réveil`.

Observation & sécurité

Observability Layer

Traces par mission (arbre décisions → actions → résultats). **Indispensable dès la V1**, sinon Nexus est une boîte noire indébogable.

Security Layer

Allégeable en interne **SAUF un point non négociable : sandbox stricte pour le code auto-généré** (Creation Engine). Un Nexus qui génère et exécute du code sans isolation peut **se détruire lui-même et l'hôte**. C'est de l'auto-préservation.

Self Repair

Périmètre strict (F5) : restart, rollback, config, état. **Jamais le code du noyau.**

Cost Monitor

Trace le **coût réel par mission / module / fournisseur / étage de cascade**. Alimente le Routing (via Evolution) et le Mission Guard (budgets). Suit aussi la **fiabilité par fournisseur** (utile pour GPT4Free).

Tableau récapitulatif

Module	Actif par défaut ?	Réveil sur
AI Gateway	 oui	—
Event Bus	 oui	—

State Graph	 oui	—
Mission Guard	 oui	—
Mission Journal	 oui	—
Capability Registry	 oui	—
Planner	non	demande de plan par Manas
Scheduler	oui (léger)	horloge / inactivité
API Gateway	oui (externe)	—
GPT4Free	non	tâche non critique explicite
Reporting Engine	non	fin de mission / demande
Observability	oui (passif)	toute action
Security Layer	oui (passif)	exécution de code
Self Repair	non	panne détectée
Cost Monitor	oui (passif)	tout appel facturé

4. Architecture API & fournisseurs

Architecture API & fournisseurs

4.1 Principe de centralisation

Tous les appels de modèles passent par **New-API** (format OpenAI-compatible unifié), lui-même piloté par l'**AI Gateway** de Nexus (voir F6).

Nexus (Routing décide) → AI Gateway (fallback, retries, CB, télémétrie) → New-API (clés, quotas) → fournisseurs

Sélection multi-fournisseurs par **coût / qualité / latence / disponibilité / criticité** — cohérent avec le Routing.

4.2 Classes de fiabilité des fournisseurs (à implémenter)

Classe	Description	Usage
local souverain	Ollama : gratuit, privé, toujours dispo	défaut des tâches simples
stable	fournisseurs fiables payés	missions normales → critiques
variable	fiabilité fluctuante	missions non critiques
best-effort	GPT4Free	exclu des chemins critiques et de la validation

Chaque appel enregistre la classe utilisée ; le Cost Monitor suit la fiabilité observée par fournisseur.

4.3 Stratégie de dégradation

Quand **tous** les fournisseurs d'une classe sont indisponibles :

1. **repli sur Ollama local** ;
2. **mode dégradé signalé dans le Mission Journal** ;
3. si la mission exige une qualité inatteignable en local → Mission Guard peut suspendre et remonter à Manas.

4.4 Hypothèse à acter

Les **quotas et clés API** sont gérés dans **New-API**. Nexus n'en connaît que les **métadonnées** (coût, limites restantes) via l'AI Gateway. Nexus ne manipule jamais les clés directement.

4.5 Recommandations d'implémentation

- **Ollama** local installé dès la V1 (défaut économique + souveraineté).
- Circuit breaker par fournisseur (seuil d'échecs consécutifs → ouverture temporaire).
- Timeout adaptatif selon l'étage de cascade (petit modèle court, raisonnement profond long).
- Télémétrie par appel : fournisseur, classe, étage cascade, tokens, coût, latence, succès/échec → Observability + Cost Monitor.

5. Schémas de données & contrats

Schémas de données & contrats

Ces schémas sont les **contrats stables** entre modules. Un agent de code doit les implémenter tels quels (les noms de champs peuvent être adaptés à la convention du langage, mais la structure est figée). Exemples donnés en JSON.

5.1 Manifeste de capacité (le contrat central)

```
{
  "id": "cap_uuid",
  "nom": "extraire_tableaux_pdf",
  "description": "Extrait les tableaux structurés d'un PDF",
  "type": "script | workflow | agent | prompt-recette | outil",
  "signature": {
    "entrees": [{"nom": "pdf_path", "type": "string"}],
    "sorties": [{"nom": "tables", "type": "array<object>"}]
  },
  "cout_typique": {"tokens": 0, "temps_ms": 1200, "argent": 0.0},
  "statut": "probatoire | eprouvee | legacy",
  "stats_usage": {
    "executions": 0,
    "succes": 0,
    "echecs": 0,
    "taux_reussite": null,
    "dernier_usage": null
  },
  "conditions_reveil": ["event:besoin_extraction_pdf"],
  "isolation": "legere | stricte | hors_sandbox",
  "provenance": "native | creee_par_nexus | fusion",
  "exemples_oracle": [
    {"entree": {}, "sortie_attendue": {}}
  ],
  "version": 1,
  "origine_fusion": null
}
```

Règles : **statut** démarre toujours à **probatoire** (P4). Promotion à **eprouvee** après N succès réels. **legacy** = remplacée par une fusion, invisible au Routing sauf échec de la fusion.

5.2 Fiche de savoir-faire (sortie de Belzébuth)

```
{
  "id": "skill_uuid",
  "contexte_declencheur": "Quand la mission demande X dans le contexte Y",
```

```

"methode": "Étapes ordonnées de la méthode qui a marché",
"pieges_connus": ["anti-pattern: ne pas faire Z"],
"cout_typique": {"tokens": 0, "temps_ms": 0, "argent": 0.0},
"taux_reussite_observe": 1.0,
"type": "reussite | echec",
"statut": "hypothese | confirmee",
"missions_source": ["mission_id_1"],
"nb_confirmations": 1
}

```

Règle : `statut` = `hypothese` tant qu'une seule mission source ; passe à `confirmee` dès la 2e mission (anti sur-généralisation).

5.3 État de mission (State Graph)

```

{
  "mission_id": "uuid",
  "objectif": "",
  "contraintes": [],
  "budgets": {"argent_max": 0.0, "temps_max_ms": 0, "appels_max": 0},
  "statut": "planifiee | en_cours | validee | abandonnee",
  "noeuds": [
    {"id": "n1", "type": "decision|action|validation", "etat": "", "parents": [], "resultat": null}
  ],
  "checklist_objectifs": [{"objectif": "", "valide": false}],
  "compteurs": {"retries": 0, "replanifications": 0},
  "reprenable_apres_crash": true
}

```

Règle : reprise après crash obligatoire. « Mission terminée » = toute la `checklist_objectifs` validée par le Validation Engine.

5.4 Entrée de Mission Journal (append-only)

```

{
  "seq": 1042,
  "timestamp": "ISO-8601",
  "mission_id": "uuid",
  "type": "decision | action | validation | evolution | cout | alerte_guard",
  "acteur": "manas | routing | execution_fabric | evolution_engine | ...",
  "payload": {},
  "signature": "hash",
  "precedent_hash": "hash"
}

```

Règle : **jamais de modification ni suppression.** Chaînage par hash (`precedent_hash`) pour l'intégrité. Les changements de l'Evolution Engine y sont **signés**.

5.5 Décision de routage

```
{
  "besoin": "",
  "type_tache": "",
  "etage_choisi": "cache|memoire|formel|outil|petit_modele|grand_modele|profond",
  "score_confiance": 0.0,
  "seuil_etage": 0.0,
  "escalade": false,
  "cout_reel": {},
  "politique_version": 7
}
```

5.6 Score de criticité (réveil de Great Sage)

```
{
  "cout_estime": 0.0,
  "irreversibilite": 0.0,
  "incertitude": 0.0,
  "criticite": 0.0,
  "seuil": 0.6,
  "great_sage_active": false
}
```

`criticite = cout_estime × irreversibilite × incertitude` (chaque facteur normalisé [0,1]).

5.7 Consensus Great Sage

```
{
  "recommandation": "",
  "niveau_confiance": 0.0,
  "objections_non_resolues": [],
  "consensus_atteint": true,
  "options_si_pas_de_consensus": []
}
```

5.8 Mission du Gymnase

```
{
  "id": "gym_mission_1",
  "objectif": "",
  "entree": {},
  "resultat_attendu": {},
  "metriques_cibles": {"cout_max": 0.0, "temps_max_ms": 0, "qualite_min": 0.0},
  "categorie": "routage | capacite | evolution"
}
```

Usage : (a) valider toute évolution de politique avant application ; (b) tester les capacités créées ; (c) prouver que la mission n°20 est mieux gérée que la n°1.

6. Structure du dépôt & plan de construction par phases

Structure du dépôt & plan de construction par phases

6.1 Structure du dépôt (matérialise le Contrat de noyau gelé, P3)

La frontière noyau/capacités est **physique** : le noyau est en **lecture seule** pour l'Execution Fabric.

```
nexus-ciel/
├── core/                                # NOYAU GELÉ — modifiable par l'humain uniquement
│   ├── manas/                          # orchestrateur (machine à états + consultations)
│   ├── event_bus/
│   ├── state_graph/
│   ├── execution_fabric/               # 3 niveaux d'isolation
│   ├── validation_engine/
│   ├── mission_journal/               # append-only, chaîné par hash
│   ├── mission_guard/                 # budgets, kill-switch
│   ├── capability_registry/           # contrat central (manifeste)
│   └── ai_gateway/                    # fallback, retries, circuit breakers
│
├── policies/                          # ÉVOLUTIF — territoire de Nexus (données, pas code système)
│   ├── routing_policy/                # versionnée ; écrite par Evolution Engine
│   └── defaults/                     # politique par défaut écrite à la main (cold start)
│
├── capabilities/                      # ÉVOLUTIF — capacités créées par Nexus (données)
│   ├── probatoire/
│   ├── eprouvee/
│   └── legacy/
│
├── memory/                           # index (embeddings, cache sémantique) — PAS un entrepôt
│
├── engines/                          # modules d'apprentissage (cycle de fond)
│   ├── belzebuth/                    # assimilation expérience → fiches
│   ├── evolution/                   # amélioration des politiques
│   ├── fusion/                      # DERNIÈRE PHASE
│   └── creation/                    # création de capacités (sandbox stricte)
│
├── support/
│   ├── planner/                      # outil de Manas
│   ├── scheduler/                   # déclenche les cycles de fond
│   ├── great_sage/                 # réveil sur criticité
│   ├── formal_reasoning/           # SAT/SMT/Z3
│   ├── reporting/
│   ├── observability/
│   ├── security/                    # sandbox stricte non négociable
│   ├── self_repair/
│   └── cost_monitor/
│
└── providers/                        # intégrations (New-API config, Ollama, GPT4Free confiné)
```

6.2 Plan de construction par phases

Ordre impératif : on ne construit pas la Fusion avant que le noyau tourne. Chaque phase produit un **système fonctionnel**, pas un demi-module.

Phase 0 — Fondations (infra)

- Event Bus + State Graph + Mission Journal (append-only) + Capability Registry (manifeste).
- Schémas de données de la page 5 implémentés comme contrats partagés.
- **Critère de sortie** : on peut créer une mission, la journaliser, la reprendre après crash.

Phase 1 — Boucle de mission minimale

- Manas Core (machine à états) + Execution Fabric (3 isolations) + Mission Guard + Validation Engine (couche déterministe seulement).
- AI Gateway + New-API + Ollama local.
- Politique de routage **par défaut écrite à la main** (pas d'apprentissage).
- **Critère de sortie** : Nexus exécute une mission simple de bout en bout, avec budgets et kill-switch.

Phase 2 — Économie & observation

- Routing System complet (cascade F3, scores de confiance, escalade).
- Cache sémantique + Memory Core (indexation + récupération).
- Observability Layer + Cost Monitor.
- Formal Reasoning Layer (niveau 3 de la cascade).
- **Critère de sortie** : on voit le coût par étage et Nexus évite d'appeler un grand modèle quand un étage moins cher suffit.

Phase 3 — Jugement & robustesse

- Validation Engine complet (3 couches + juge fournisseur différent + budgets N/M).
- Great Sage (réveil sur criticité, 5 perspectives dont mémoire non-LLM).
- Self Repair + Security Layer durci.
- **Critère de sortie** : pas de boucle infinie, décisions critiques confrontées, récupération sur panne.

Phase 4 — Apprentissage (le « sommeil »)

- Scheduler + cycle de digestion.
- Belzébut (fiches de savoir-faire, hypothèse → confirmée).
- Politique d'oubli du Memory Core (consolidation / déclassement / archivage).
- **Critère de sortie** : Nexus produit des fiches réutilisables et la mémoire ne gonfle pas indéfiniment.

Phase 5 — Création de capacités

- Capability Creation Engine (oracle avant code, revue par autre modèle, sandbox stricte, probatoire).
- Gymnase de missions opérationnel.
- **Critère de sortie** : Nexus crée un outil manquant, le teste sur exemples réels, le promeut après N succès.

Phase 6 — Évolution mesurée

- Evolution Engine (seul propriétaire du routage, F1) + validation sur Gymnase + rollback + étalon d'origine.
- **Critère de sortie** : on **prouve** (Gymnase) qu'une nouvelle politique fait mieux avant de l'appliquer.

Phase 7 — Fusion (dernier, spéculatif)

- Capability Fusion Engine + politique de dépréciation (legacy).
- **Critère de sortie** : le Registry se **simplifie** au lieu de gonfler.

6.3 Graine de connaissances initiale (cold start)

Livrer Nexus avec un **pack initial** : politique de routage par défaut + quelques fiches de savoir-faire de base + missions du Gymnase. Atténue le démarrage à froid.

7. Guide de l'agent de code (Cursor / LLM)

Guide de l'agent de code (Cursor / LLM)

Cette page s'adresse directement à l'IA qui va écrire le code. À coller en tête de contexte / en règles de projet.

Règles absolues (ne jamais violer)

1. **Ne jamais improviser une décision d'architecture.** Si elle n'est pas dans ce document, signale le trou, propose 2 options, attends. C'est le but explicite du document.
 2. **Contrat de noyau gelé** : le code sous `core/` ne doit **jamais** être modifiable par le système lui-même. Aucune capacité générée ne touche `core/`.
 3. **Sandbox stricte, sans réseau par défaut, pour tout code auto-généré** (Creation Engine). Non négociable.
 4. **Sources de vérité uniques** : ne jamais dupliquer capacités / historique / état. Le Memory Core indexe, il n'entrepasse pas.
 5. **Mission Journal append-only** : jamais d'update/delete. Chaînage par hash.
 6. **Pas de juge unique LLM** : validation déterministe d'abord, LLM ensuite, juge d'un **autre fournisseur** pour le critique.
 7. **Budgets partout** : retries `N`, replanifications `M`, budgets coût/temps/appels. Aucune boucle sans borne.
 8. **Apprentissage uniquement en cycle de fond** (Scheduler, inactivité). Jamais pendant une mission.
 9. **Seul l'Evolution Engine modifie les politiques de routage.** Le Routing les applique en lecture seule.
 10. **Tout nouveau** = `probatoire` jusqu'à N succès réels.
-

Ordre de construction

Suivre strictement les **Phases 0 → 7** de la page 6. Ne pas commencer un module d'une phase avant que la précédente passe son critère de sortie. La Fusion (Phase 7) est la dernière et la plus risquée.

Conventions attendues

- Implémenter d'abord les **schémas de la page 5** comme types/contrats partagés : tout le reste en dépend.
- Chaque module expose ses **conditions de réveil** (abonnements Event Bus) plutôt que d'être appelé en dur.
- **Télémetrie systématique** vers Observability + Cost Monitor à chaque action et chaque appel de modèle.

- Tests : chaque capacité et chaque module a un **oracle** (exemples entrée/sortie réels) avant d'être considéré fini.
- Stack suggérée (à confirmer par l'humain avant Phase 0) : langage unique pour le noyau, New-API + Ollama pour les modèles, Docker pour les 3 niveaux d'isolation, Z3 pour le Formal Reasoning.

Checklist « définition of done » par module

- ☐ Rôle et frontières respectés (aucun chevauchement réintroduit — revoir F1→F7).
- ☐ Contrat d'entrée/sortie conforme aux schémas page 5.
- ☐ Conditions de réveil déclarées (activation paresseuse).
- ☐ Télémétrie branchée.
- ☐ Cas limites documentés de la page 2 gérés explicitement.
- ☐ Budgets/bornes en place (le cas échéant).
- ☐ Tests avec oracle réel qui passent en sandbox appropriée.

Ce qui reste à trancher par l'humain (points ouverts)

Le document source était tronqué à la fin de la section S6. À confirmer avant/pendant le build :

- Stack technique précise (langages, runtime, base de données pour State Graph / Journal / Registry).
- Valeurs numériques par défaut : seuil de criticité (proposé 0.6), N (promotion probatoire), M (replanifications max), seuils de confiance par étage.
- Contenu exact de la **graine de connaissances initiale** (S6, tronqué).
- Format détaillé des 3 rapports (mission, évolution, coûts).

Quand l'un de ces points est tranché, l'ajouter ici et le figer, comme pour F1→F7.